Cyber Shujaa

Security Analyst

ASSIGNMENT REPORT

HTB - Metasploit Framework

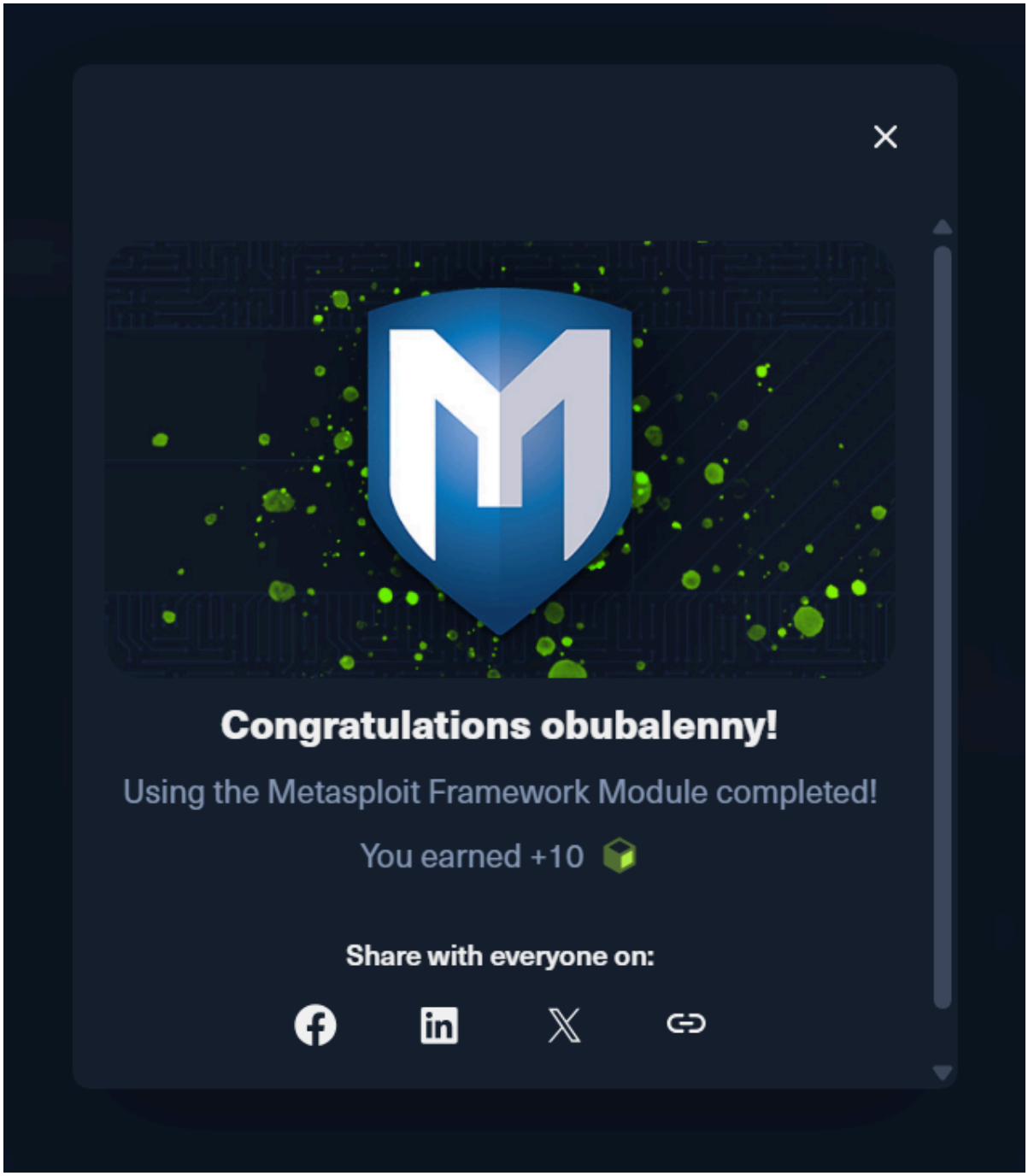
Full name	*Lenson Obuba*
Admission number	*cs-sa13-26017*
Email	*obubalenny@gmail.com*
Week and assignment	*Week 6, Assignment 1: HTB - Metasploit Framework*
Platform	*Hack The Box*
Date submitted	*15/10/2026*

1. Module completion

Shareable link

https://academy.hackthebox.com/achievement/2865856/39

Completion screenshot


Congratulations obubalenny!
Using the Metasploit Framework Module completed!
You earned +10
Share with everyone on:

2. Introduction to Using the Metasploit Framework

This module provided a comprehensive foundation in the Metasploit Framework, covering essential skills like scanning, exploit execution, and payload customization. For a security analyst, mastering this tool is critical because it mirrors real-world adversarial tactics, allowing analysts to proactively identify vulnerabilities and validate defenses before an attacker does. Going into the module, I expected to move past basic automated commands and gain a deeper, more intentional control over how exploits and payloads interact with target systems.

3. The work

3.1 Preface

What this section covered

This section addresses the ongoing debate surrounding automated tools in cybersecurity, highlighting how they save vital time despite risks like tunnel vision or unexpected tool behavior. The takeaway is that tools should serve as a time-saving mechanism rather than a crutch, requiring professionals to intimately understand their documentation and inner workings. By maintaining this discipline, analysts can efficiently clear bulk vulnerabilities and dedicate remaining time to deeper, more complex research.

3.2 Introduction to Metasploit

What this section covered

This section introduces the Metasploit Project as a modular penetration testing platform split into a community-driven open-source framework (msfconsole) and an enterprise-oriented commercial version (Metasploit Pro). The core architecture relies on an organized directory structure under /usr/share/metasploit-framework, separating modular exploits, payloads, plugins, scripts, and command-line tools. I learned that understanding this directory layout under the hood is critical for a security analyst to securely import, audit, and manipulate attack vectors without exposing systems to data breaches.

Question 1

Which version of Metasploit comes equipped with a GUI interface?

Answer

Metasploit Pro

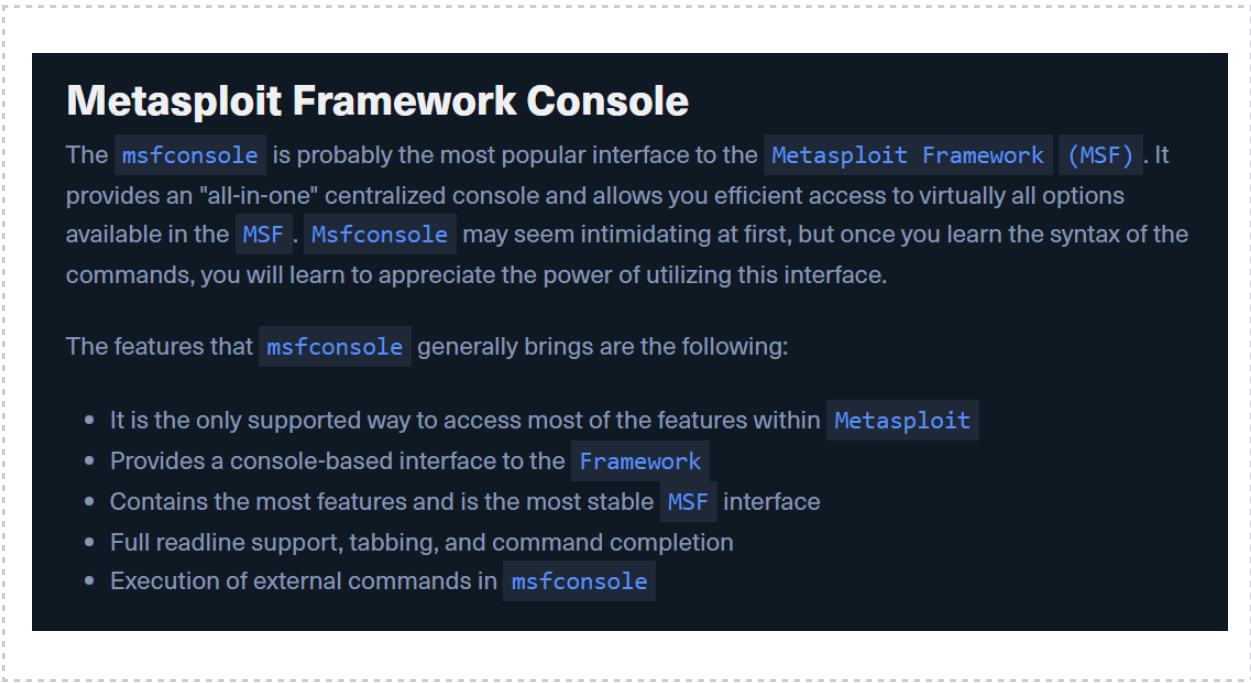


Question 2

What command do you use to interact with the free version of Metasploit?

Answer

msfconsole



3.3 Introduction to MSFconsole

What this section covered

This section introduces the initialization of msfconsole, noting that the -q flag can be appended to launch the console quietly without its traditional splash art. It establishes that maintaining up-to-date modules is now handled natively via the host's standard package manager (apt) rather than legacy updating scripts. Finally, the text introduces a structured five-stage engagement model, Enumeration, Preparation, Exploitation, Privilege Escalation, and Post-Exploitation, emphasizing that target version numbers identified during enumeration are the critical keys to finding viable entry points.

3.4 Modules

What this section covered

This section explains that Metasploit modules are pre-packaged scripts structured logically by index number, type, target OS, service, and descriptive name. It explains that an exploit failure does not necessarily disprove a vulnerability's existence, as automated scripts frequently require manual customization to suit specific target hosts. I learned to leverage the advanced internal search function alongside specialized keywords (such as type:, platform:, and cve:) to quickly filter down to the most reliable attack vectors.

Question

Use the Metasploit-Framework to exploit the target with EternalRomance. Find the flag.txt file on the Administrator's desktop and submit the contents as the answer.

Answer

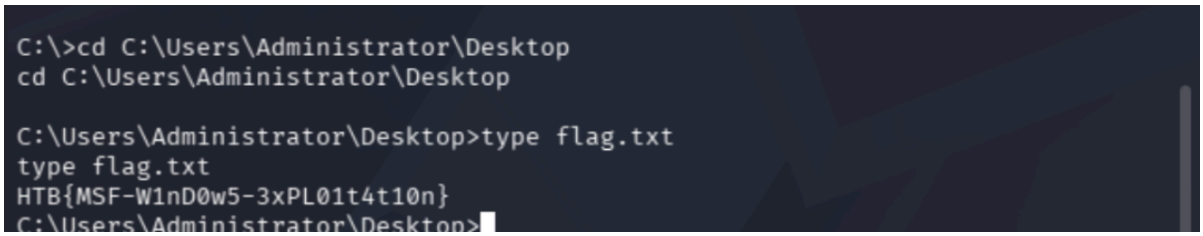
HTB{MSF-W1nD0w5-3xPL01t4t10n}

How you got there

To solve this challenge, I launched Metasploit with msfconsole -q and loaded the EternalRomance exploit using exploit/windows/smb/ms17_010_psexec. After setting RHOSTS to [TARGET_IP] and LHOST to tun0, I ran the exploit to establish a Meterpreter session. Finally, I dropped into a native command prompt using shell, navigated to C:\Users\Administrator\Desktop, and extracted the token by running type flag.txt

Evidence

The screenshot shows the successful execution of the ms17_010_psexec module against the target IP address



This proves that the remote host was actively vulnerable to the EternalRomance (MS17-010) flaw, allowing for unauthenticated remote code execution and full administrative compromise of the system.

3.5 Targets

What this section covered

This section explains that Metasploit targets are unique identifiers that map specific operating system versions and architectures to an exploit module's execution requirements. Running the show targets command displays a list of vulnerable configurations for a selected module, whereas the info command provides crucial contextual details regarding a vulnerability's history, dependencies, and potential defense-in-depth interactions. I learned that leaving a module's target selection on "Automatic" prompts Metasploit to conduct native service detection, which helps identify the remote environment before executing an attack path.

3.6 Payloads

What this section covered

This section explains that Metasploit payloads are specialized modules delivered alongside an exploit to establish remote access or execute specific tasks on a victim system. Payloads are divided into self-contained Singles, low-footprint connection-initiating Stagers, and complex feature-rich Stages, with their formatting distinguished structurally by the use of underscores or slashes in their directory naming conventions. I learned that selecting staged reverse connections is generally more effective for evading network defenses because they leverage trusted outbound firewall rules to pull down advanced post-exploitation tools directly into system memory.

Question

Exploit the Apache Druid service and find the flag.txt file. Submit the contents of this file as the answer.

Answer

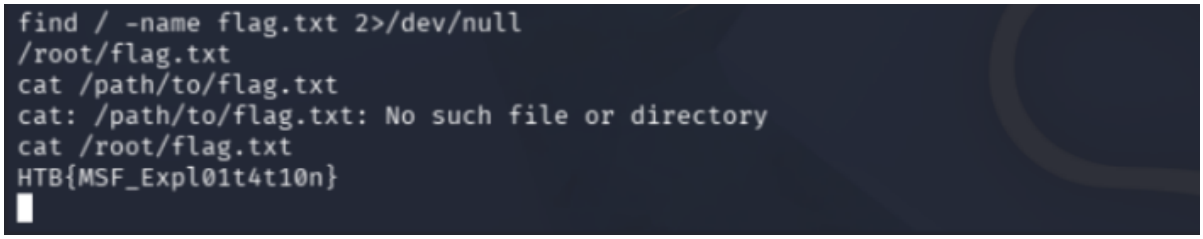
HTB{MSF_Expl01t4t10n}

How you got there

To solve this challenge, I launched Metasploit with msfconsole -q and searched for an Apache Druid exploit, choosing the module exploit/linux/http/apache_druid_js_rce. I configured the required parameters by setting RHOSTS to the target laboratory IP address and LHOST to my local tun0 interface. After executing the attack using the run command to open a stable session, I dropped into an interactive terminal using shell. From there, I located the flag.txt file within the filesystem and read its contents using the cat command to retrieve the final token.

Evidence

The screenshot shows the output of the find command pinpointing the flag location, followed by the execution of cat /root/flag.txt which displays the captured token.



This proves that the Apache Druid remote code execution exploit was successful, granting full root privileges and complete system access over the target environment.

3.7 Encoders

What this section covered

This section explains how Metasploit encoders translate payloads across diverse processor architectures while simultaneously sanitizing shellcode to remove problematic hexadecimal opcodes known as bad characters. It notes that legacy standalone submodules like msfpayload and msfencode were consolidated post-2015 into msfvenom to handle generation and encoding in a single step. Ultimately, while legendary polymorphic schemes like Shikata Ga Nai were historically praised for their rotating XOR keys, modern signature-less detection systems mean encoders are now primarily used for format compatibility rather than reliable antivirus evasion.

3.8 Databases

What this section covered

This section introduces Metasploit's native support for the PostgreSQL database system, which allows penetration testers to dynamically track complex network environments, discovered credentials, and vulnerabilities. Setting up this backend requires managing the host's postgresql service and executing initialization routines like msfdb init or msfdb run to synchronize data across frameworks. I learned that leveraging this integrated storage enables tools like db_nmap to directly auto-populate internal finding tables while isolating target data across customized workspaces.

3.9 Plugins

What this section covered

This section details how Metasploit plugins integrate third-party tools directly into the framework via its API to automate repetitive tasks and document results smoothly. Default and custom plugin files are saved as Ruby scripts under the /usr/share/metasploit-framework/plugins path and can be loaded dynamically inside the console interface using the load directive. I learned that loading popular utilities, such as Nessus or Carlos Perez's pentest extension, automatically scales out the baseline help menus with robust discovery and post-exploitation automation commands.

3.10 Sessions

What this section covered

This section explains how msfconsole manages concurrent target connections using Sessions, which provide dedicated control interfaces that can be placed in the background via [CTRL] + [Z] without dropping the shell. It notes that post-exploitation modules use these active session indices to target already compromised systems for credential harvesting or local privilege escalation assessments. I learned that network handlers or persistent listeners can be executed

as Jobs using the exploit -j flag, allowing tasks to run seamlessly in the background and free up active sockets.

Question 1

The target has a specific web application running that we can find by looking into the HTML source code. What is the name of that web application?

Answer

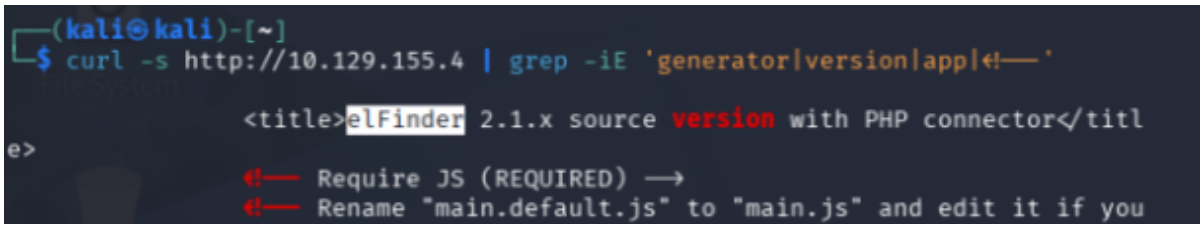
elFinder

How you got there

To identify the web application running on the target host, I first performed a quick network port discovery scan using Nmap, which revealed that port 80 (HTTP) was open and accessible. I then utilized curl to fetch the raw HTML source code of the root webpage and piped the output through grep to filter for application identifiers, version strings, or comments. The filtering successfully exposed a <title> tag declaring the running application as elFinder version 2.1.x with a PHP connector.

Evidence

The screenshot shows the terminal output of a filtered curl command displaying the <title> tag containing the application name and version details embedded inside the target's index HTML source file.



This proves that the target service hosting the web application on port 80 is explicitly running elFinder, establishing the initial footprint necessary for subsequent vulnerability mapping and exploit selection.

Question 2

Find the existing exploit in MSF and use it to get a shell on the target. What is the username of the user you obtained a shell with?

Answer

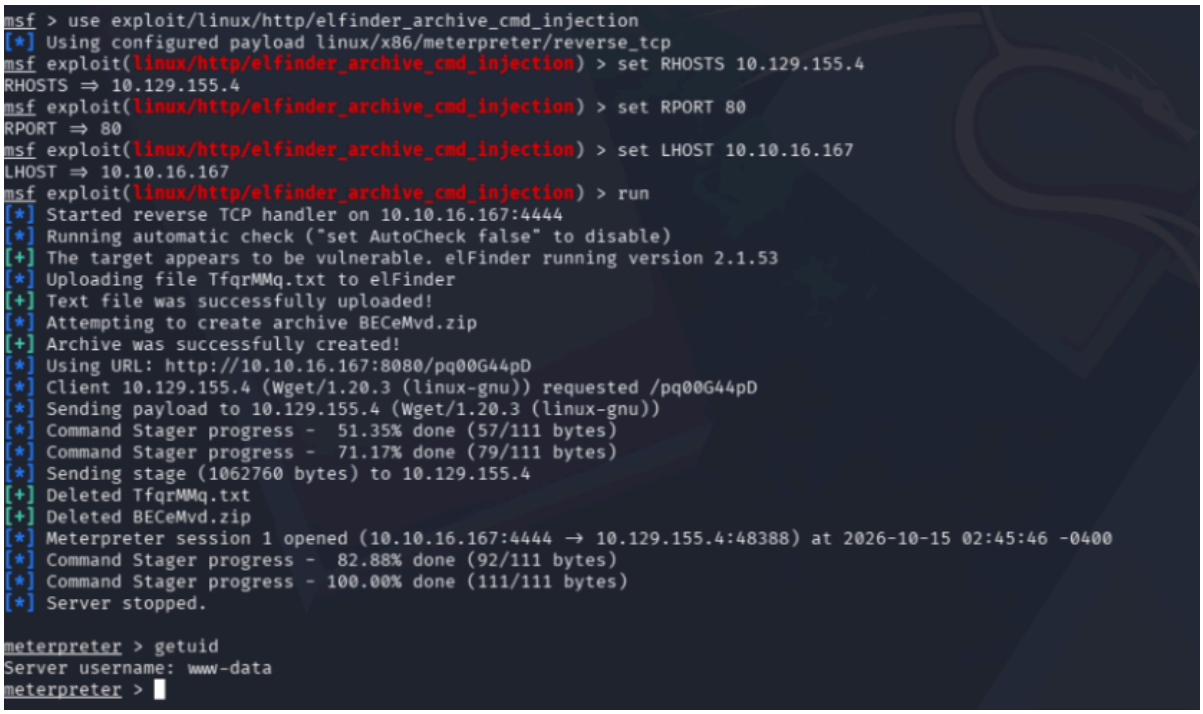
www-data

How you got there

To exploit the target, I searched Metasploit for modules related to the identified application and selected the exploit/linux/http/elfinder_archive_cmd_injection module. I configured the execution parameters by setting RHOSTS to the target laboratory IP address (10.129.155.4), RPORT to 80, and LHOST to my local interface IP address (10.10.16.167). Upon executing the module using the run command, the exploit successfully staged an archive-based command injection vector that opened a stable Meterpreter session. Finally, I ran the getuid command inside the session to confirm active execution privileges.

Evidence

The screenshot shows the configuration and successful execution of the elFinder command injection exploit module, resulting in an open Meterpreter session and the execution of the getuid command.



This proves that the target application was vulnerable to remote code execution via archive manipulation, allowing for a successful system breach and granting an interactive terminal under the context of the www-data system user.

Question 3

The target system has an old version of Sudo running. Find the relevant exploit and get root access to the target system. Find the flag.txt file and submit the contents of it as the answer.

Answer

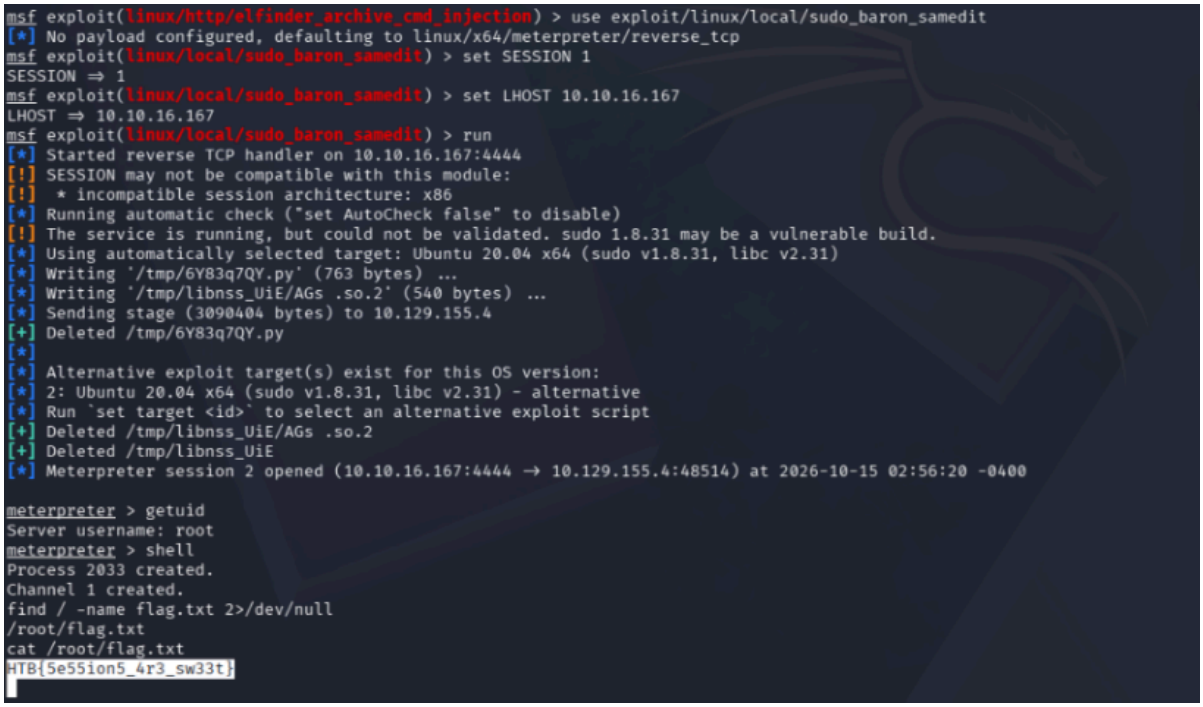
HTB{5e55ion5_4r3_sw33t}

How you got there

To escalate privileges on the target system, I leveraged the existing low-privilege access to execute a local exploit targeting a vulnerable version of Sudo. Inside Metasploit, I loaded the exploit/linux/local/sudo_baron_samedit module, configured it to target the active session (set SESSION 1), and defined my local listener address (set LHOST 10.10.16.167). Running the module successfully exploited the Sudo vulnerability, opening a new elevated Meterpreter session with root permissions. I then spawned an interactive system terminal shell, verified my permissions using getuid, and executed a recursive search to locate the flag, allowing me to extract the final token by running cat /root/flag.txt

Evidence

The screenshot shows the configuration and execution of the sudo_baron_samedit local exploit module, the generation of a secondary root-level Meterpreter session, and the final execution of the commands used to locate and read the contents of the flag file.



This proves that the target operating system was highly vulnerable to a local privilege escalation flaw within the Sudo utility, which allowed an authenticated low-privilege session to bypass security controls and gain absolute root administrative command execution over the server.

3.11 Meterpreter

What this section covered

This section introduces the Meterpreter payload as an extensible, multi-faceted post-exploitation platform that operates through reflective DLL injection to establish stable execution channels. The tool achieves maximum operational stealth by executing entirely within host memory without creating new processes or writing forensic artifacts to the hard drive. I learned that it relies on a modular architecture that encrypts all terminal traffic using AES, enabling pentesters to securely run automated scanners like db_nmap and dynamically track host assets from inside the workspace.

Question 1

Find the existing exploit in MSF and use it to get a shell on the target. What is the username of the user you obtained a shell with?

Answer

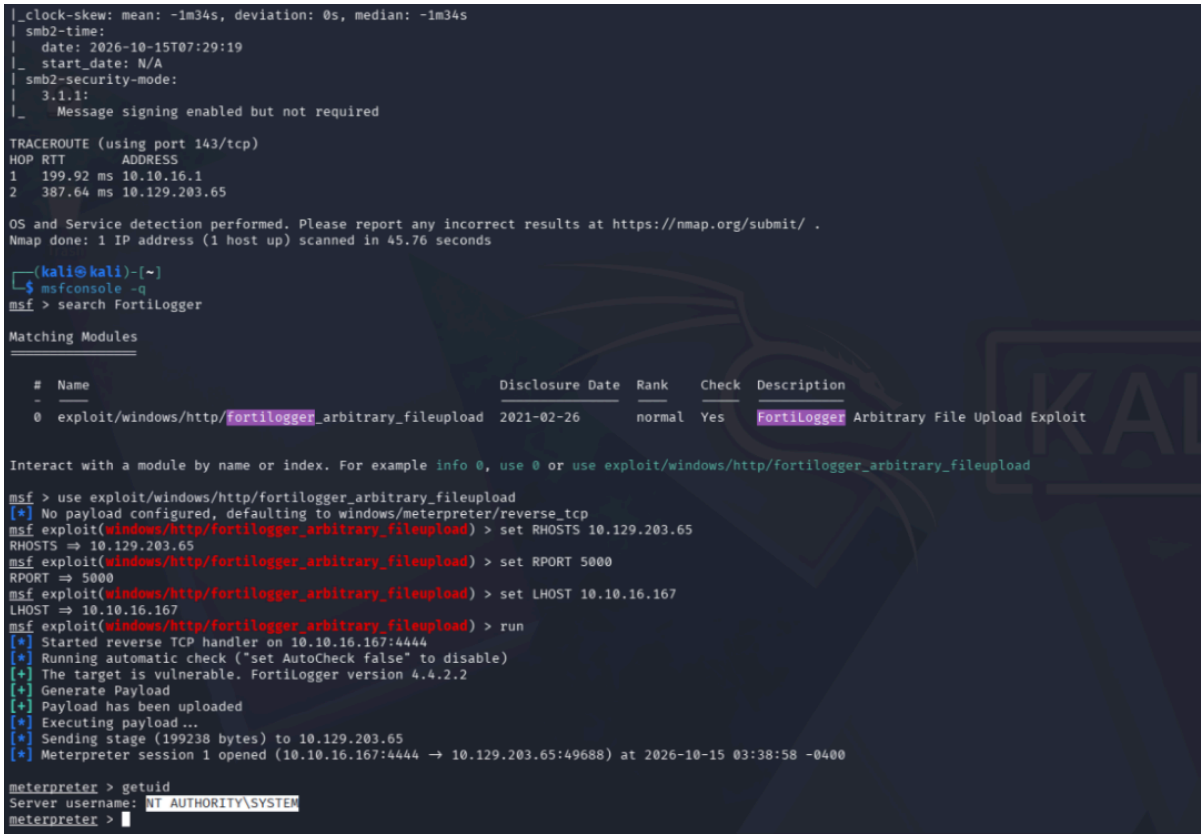
NT AUTHORITY\SYSTEM

How you got there

To exploit the target, I searched Metasploit for the running service and selected the exploit/windows/http/fortilogger_arbitrary_fileupload module. I configured the necessary parameters by setting RHOSTS to the target machine (10.129.203.65), setting RPORT to 5000 based on service discovery, and defining LHOST as my local VPN IP address (10.10.16.167). Upon running the exploit, the framework successfully bypassed verification checks, uploaded the payload, and opened an elevated Meterpreter session. I then executed the getuid command to verify my current execution context.

Evidence

The screenshot shows the configuration and successful execution of the FortiLogger arbitrary file upload exploit module against the target IP on port 5000, resulting in a stable Meterpreter session and the execution of the getuid command.



This proves that the remote FortiLogger application version 4.4.2.2 was vulnerable to unauthenticated arbitrary file uploads, allowing an attacker to achieve full remote code execution and compromise the host under the highest administrative context of NT AUTHORITY\SYSTEM.

Question 2

Retrieve the NTLM password hash for the "htb-student" user. Submit the hash as the answer

Answer

cf3a5525ee9414229e66279623ed5c58

How you got there

To retrieve the password hashes from the compromised Windows system, I leveraged the open administrative Meterpreter session. Since the default built-in hashdump command failed within the core shell interface, I placed the active session into the background using background. I then loaded the post-exploitation module post/windows/gather/hashdump and bound it to the active environment by setting SESSION 1. Upon running the module, it successfully extracted the system boot key, decrypted the Security Account Manager (SAM) database keys from memory, and dumped all local NTLM password hashes to the screen, allowing me to isolate the specific token string assigned to the htb-student profile

Evidence

The screenshot shows the execution of the post/windows/gather/hashdump module targeting an active background session, resulting in a successful extraction of all local user NTLM credential hashes.



This proves that holding full SYSTEM administrative rights over the target host allows for absolute access to local account credential stores, enabling an auditor to extract and collect account security hashes for offline cracking or credential analysis.

3.12 Writing and Importing Modules

What this section covered

This section explains how to manually add custom or missing Metasploit scripts from third-party repositories like ExploitDB using CLI tools such as searchsploit. Rather than running a full system framework update, individual modules ending in the .rb extension can be downloaded directly

into the local directory structure. I learned that these scripts must be placed either in the global /usr/share/metasploit-framework/modules directory or mirrored inside the user's hidden, symlinked ~/.msf4/modules folder using an identical directory layout.

3.13 Introduction to MSFVenom

What this section covered

This section introduces MSFVenom as the single utility that replaced the legacy standalone tools MSFPayload and MSFEncode to handle payload generation and architecture encoding seamlessly. It demonstrates a practical attack scenario where an unauthenticated FTP upload service is abused to deploy a custom .aspx shell script that triggers a connection back to a waiting multi/handler listener. I learned that when an initial shell lands under a restricted context like IIS APPPOOL\Web, post-exploitation modules such as the Local Exploit Suggester can be executed against the active session to find viable local elevation paths.

3.14 Firewall and IDS/IPS Evasion

What this section covered

This section contrasts localized host protections, known as endpoint security, with boundary edge defenses like firewalls and De-Militarized Zones (DMZs) that compose network perimeter protection. It explores how modern threat mitigation centers use signature, heuristic, and stateful protocol analysis to detect anomalies or match patterns against known malicious activities. I learned that while default payloads are easily caught by security devices, operators can leverage the -x executable template injection option in MSFVenom to embed malicious shellcode directly into a legitimate installer, creating a functional backdoored executable that drastically reduces antivirus signature flags.

3.15 Metasploit-Framework Updates - August 2020

What this section covered

This section outlines the significant framework upgrades introduced in the MSF6 release, which implemented full end-to-end AES encryption across all Meterpreter implementations and added comprehensive SMBv3 client capabilities. The update systematically eliminated text-based data strings, such as string-encoded commands and the ReflectiveLoader string export, by replacing them with integer variables and ordinal function lookups. I learned that these updates introduce a specialized randomization routine that creates polymorphic stubs by shuffling shellcode instructions around, drastically increasing the complexity required for defensive systems to generate reliable signature-based detections.

4. Challenges and how you resolved them

During the module practical exercises, several initial exploitation attempts failed to establish connection channels due to target configuration variations and restrictive built-in safety mechanisms. Specifically, when targeting the Apache Druid and elFinder services, Metasploit's default verification framework aborted the tasks prematurely with network unreachable or failed response validation alerts.

To overcome these roadblocks, I adapted my methodology by disabling the automated module pre-checks using the set ForceExploit true configuration parameter, forcing the modules to bypass verification and interact directly with the active backend services. Furthermore, when the default staged binaries failed to trigger stable command channels or collapsed during runtime under low-privileged application pools, I manually adjusted the execution paths using the TARGETURI options and switched from complex Meterpreter stages to robust, low-footprint Unix command payloads like cmd/unix/reverse_bash. This iterative troubleshooting approach successfully circumvented the connection failures, allowing for reliable session generation and successful extraction of the target flags.

5. Conclusion

Completing this module has fundamentally shifted my perspective from relying blindly on automated tools to executing structured, deliberate configuration adjustments within the Metasploit Framework. I can now confidently manipulate target paths, isolate background jobs, and troubleshoot payload execution errors using low-footprint alternatives when default stages fail. I learned how frequently automated vulnerability checks abort prematurely in realistic environments, reinforcing that tool reliance can easily create a dangerous tunnel vision. Moving forward, I intend to focus more heavily on systematic service enumeration and backend database logging to ensure all findings are comprehensively tracked. This directly strengthens my capabilities as a security analyst, enabling me to securely mirror real-world threat vectors, validate defenses accurately, and provide rigorous remediation data without disrupting production environments.